

CS & IT ENGINEERING

COMPILER DESIGN

PARSING

Lecture No.- 10



By- Venkat sir

Recap of Previous Lecture



Topic

?? LL(1) Parsing

LL(1) Grammar detection

LL(1) Parsing Table Construction



Topics to be Covered



Topic

??

Left Recursion Removal

Topic

??

Left factoring

Topic

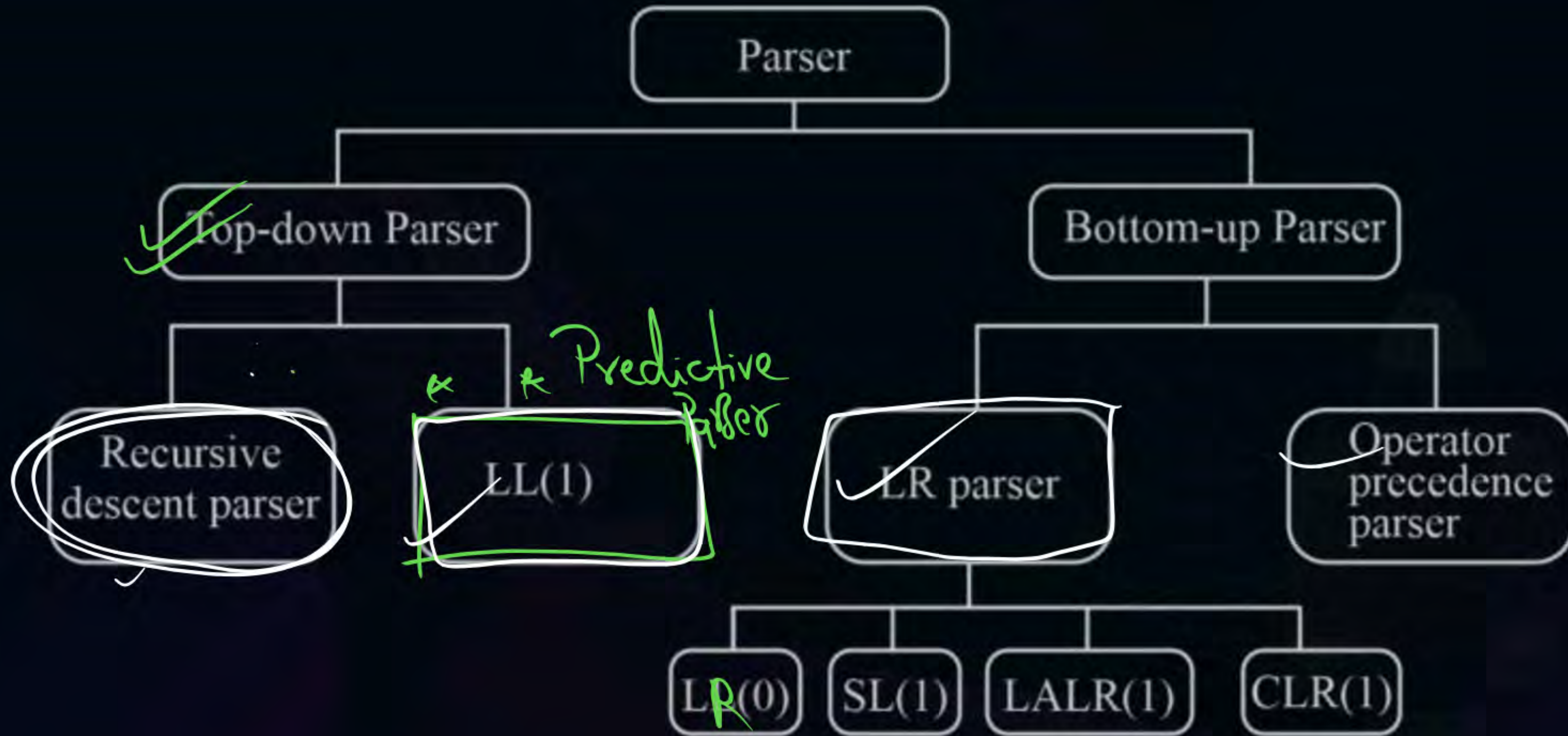
??

LL(1) Parsing Algorithm

Recursive Descent Parsing



Topic : PARSER





Topic : LL(1) Parsing



$$E \rightarrow E + T \mid T$$

① Left Recursive Grammar is not LL(1)

But removal of Left Recursion may result LL(1) grammar

ba^* not LL(1)

{ Removal of Left Recursion }

① $S \rightarrow \underline{S}a/b$

$\{ \underline{ba}^* \}$

$\checkmark S \rightarrow \underline{b} \underline{S'}$
 $\overset{a^*}{S'} \rightarrow \underline{a} \underline{S'} / \epsilon$
 $\{ a \mid \beta \}$
 $\{ LL(1) \}$

② $E \rightarrow \underbrace{E \overset{R}{+} T}_{\underbrace{\quad}_{T \rightarrow a}} \mid \overset{N.R.}{T}$

③ $A \rightarrow A\alpha / \underline{\beta}$

$A \rightarrow \beta A'$

$A' \rightarrow \alpha A' / \epsilon$

$E \rightarrow \underline{T} E'$

$E' \rightarrow \underbrace{+ T}_{\underbrace{\quad}_{T \rightarrow a}} E' / \epsilon$

$T \rightarrow a$

Left Recursion Removal

$$A \rightarrow \underline{A} \alpha_1 \mid \underline{A} \alpha_2 \mid \dots \mid \underline{A} \alpha_n \mid \beta_1 \mid \beta_2 \mid \dots \mid \beta_m$$



$$A \rightarrow \beta_1 A' \mid \beta_2 A' \mid \dots \mid \beta_m A'$$

$$A' \rightarrow \alpha_1 A' \mid \alpha_2 A' \mid \dots \mid \alpha_n A' \mid \epsilon$$

①

$$S \rightarrow S \underline{a} / S \underline{b} / \underline{c} / d / e$$

$$S \rightarrow cs' / ds' / es'$$

$$S' \rightarrow as' / bs' / e$$



Topic : LL(1) Parsing

Left Recursion

Indirect L.R

Ex:-

not LL(1)

$A \rightarrow \textcircled{B} a | b$
 $B \rightarrow \underline{A} b | a$

$A \rightarrow A b a | b$

Direct L.R

$A \rightarrow A \alpha | \beta$

not LL(1)



Topic : LL(1) Parsing

Left factoring

① not LL(1)
 $S \rightarrow \underline{a}b | \underline{a}c$

L.F

$\checkmark S \rightarrow \underline{a} \underline{S'}$
 $\checkmark S' \rightarrow b | c$

$$\left\{ \begin{array}{l} xy + xz \\ x(\underline{y+z}) \end{array} \right\}$$

② $S \rightarrow \underline{iEtS} / \underline{iEtSeS} / a$ } Perform Left factoring on given grammar

~~not LL(1)~~

$\Downarrow$

$\left\{ \begin{array}{l} S \rightarrow iEtS S' / a \\ S' \rightarrow \epsilon / es \end{array} \right\}$

Q) Which of the following is sufficient to convert CFG into $LL(1)$ grammar?



a) Removal of Left Recursion

b) Left factoring

c) both a & b

d) none



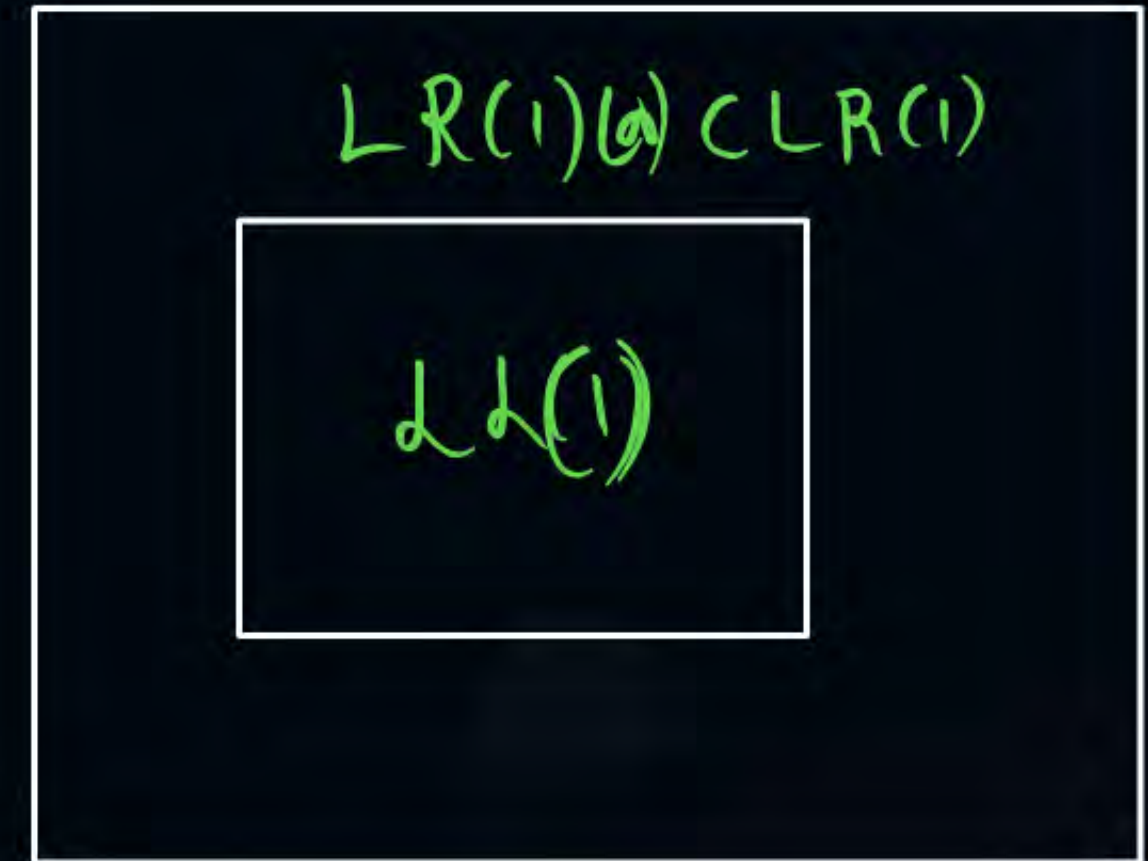
(1) $S \rightarrow \frac{aSa}{a} / \frac{bSb}{b} / \frac{c}{c}$ Given Grammar is
(2)

(a) $LL(1)$ but not $CLR(1)$

(b) $CLR(1)$ but not $LL(1)$

(c) $LL(1)$ and $CLR(1)$

(d) none



(Q) which of the following is true?

- (a) Every $LL(1)$ is $LR(0)$ grammar
- (b) Every $LL(1)$ grammar is $SLR(1)$ grammar
- (c) Every $LL(1)$ grammar is $LALR(1)$ grammar
- (d) none



Topic : LL(1) Parsing

L-R L.M.D L.A Symbol



$T.C = O(n)$

$\begin{cases} S \rightarrow AB \\ A \rightarrow a|b \\ B \rightarrow b|a \end{cases}$

LL(1) Table

	a	b	\$
S	<u>$S \rightarrow AB$</u>		
A	<u>$A \rightarrow a$</u>	$A \rightarrow b$	
B	$B \rightarrow a$	<u>$B \rightarrow b$</u>	

stack i/p
① If $x = a = \$$ then accepted.

② If $x = a \neq \$$ pop x from stack and Increment L-1

③ If x is Nonterminal

$[x, a] = x \rightarrow uvw$

Replace x by uvw in Reverse order

④ If entry is Blank then Syntax error generated



Topic : LL(1) Parsing

$S \rightarrow AB$
 $A \rightarrow a$
 $B \rightarrow b$

Stack

① $\boxed{\$ | (S)}$

② $\boxed{\$ | B | (A)}$

③ $\boxed{\$ | B | (a)}$

④ $\boxed{\$ | B}$

⑤ $\boxed{\$ | (b)}$

⑥ $\boxed{\$}$

input

$\boxed{a | b | \$}$
↑

$\boxed{a | b | \$}$
↑

$\boxed{(a) | b | \$}$

$\boxed{b | \$}$

$\boxed{b | \$}$

$\boxed{\$}$

Action

$S \rightarrow AB$ ←

$A \rightarrow a$

pop a

$B \rightarrow b$

pop b

accepted



Topic : LL(1) Parsing



- ① LL(1) Table Construction
- ② LL(1) grammar detection
- ③ Left Recursion Removal
- ④ Left factoring
- ⑤ LL(1) Parsing Algorithm.

{ Recursive Descent Parser }

2 * 3 * 4 * 5

$E \rightarrow \boxed{\text{num}} T$
 $T \rightarrow \underline{*} \text{num} T \mid \epsilon$

main()

{

E();

}

2 * 3 * 4 * 5

LA

E()

{ if (lookahead == num)

{ match(); ✓

T();

} else error();

→ if (lookahead == \$)
 { print("Success");
 } else error();

T()

{ if (lookahead == *)

{ match();

if (lookahead == num)

{ match()

T();

} else error

else
 return null,


```
match()
```

```
{
```

```
    lookahead = next token;
```

```
}
```

```
error()
```

```
{
```

```
    print("Syntax error");
```

```
}
```




THANK - YOU